



# BİLGİSAYAR AĞLARI DERSİ DÖNEM PROJESİ TEKNİK RAPORU

**Proje Adı:**

NetProbe: UDP Üzerinde Stop-and-Wait Tabanlı Güvenilir Veri Aktarım Protokolü Mimari  
Tasarımı ve Performans Analizi

**Grup No:15**

## STAJ YAPAN ÖĞRENCİNİN

**Adı ve Soyadı** :Gülizar ALTINTAŞ / Şeymanur EVİN  
**Numarası** :19360859019 / 21360859067  
**Fakültesi** :Mühendislik ve Doğa Bilimleri  
**Bölümü** :Bilgisayar Mühendisliği

2025/2026 ÖĞRETİM YILI

## 1. Giriş (Introduction)

Bilgisayar ağlarında taşıma katmanı (Transport Layer) servisleri, uygulama katmanından gelen verilerin uçtan uca iletim stratejisini belirler. Modern internet mimarisi temelde iki ana taşıma katmanı felsefesi üzerine kuruludur: Bağlantı yönelimli, akış kontrolü ve hata denetimi sunan güvenilir **TCP (Transmission Control Protocol)** ile bağlantısız, durum takibi yapmayan ancak minimal başlık yükü sunan **UDP (User Datagram Protocol)**.

TCP protokolü; tıkanıklık kontrolü (congestion control), kayıp paketlerin yeniden iletimi (ARQ) ve bit hatalarının tespiti süreçleriyle veri bütünlüğünü %100 garanti altına alırken, bağlantı kurulumundaki üçlü el sıkışma (three-way handshake) gecikmeleri ve dinamik pencere daraltma mekanizmaları nedeniyle zaman-kritik uygulamalarda (canlı yayın, siber güvenlik log aktarımı, anlık telemetri) yüksek varyanslı gecikmelere (jitter) neden olur. Öte yandan UDP, ağır durumunu ve paketlerin ulaşp ulaşmadığını umursamaksızın datagramları en hızlı şekilde hat üzerine bırakır. Ancak bu hız avantajının karşılığında paket kayıpları, bit seviyesinde bozulmalar veya paketlerin sırasının değişmesi (out-of-order) gibi siber güvenlik ve veri bütünlüğü açısından kabul edilemez riskleri barındırır.

## 2. Problem Tanımı (Problem Definition)

Gerçek dünya ağ altyapıları ideal değildir; yönlendirici (router) kuyruklarının dolması, elektromanyetik parazitler ve hat üzerindeki fiziksel olumsuzluklar nedeniyle paketler kayba uğrayabilir veya bit seviyesinde deforme olabilir. Bu projenin temel problemi; UDP'nin sunduğu düşük gecikmeli veri iletim yeteneklerinden ve soket esnekliğinden taviz vermeden, kararsız ve yüksek kayıplı ağ ortamlarında bile verinin karşı tarafa kayıpsız, bozulmasız ve tam sıralı ulaştırılmasını sağlayacak bir **Uygulama Katmanı Güvenilirlik Protokolü** tasarlamaktır.

Projenin mühendislik kısıtları ve çözülmesi gereken alt problemleri şunlardır:

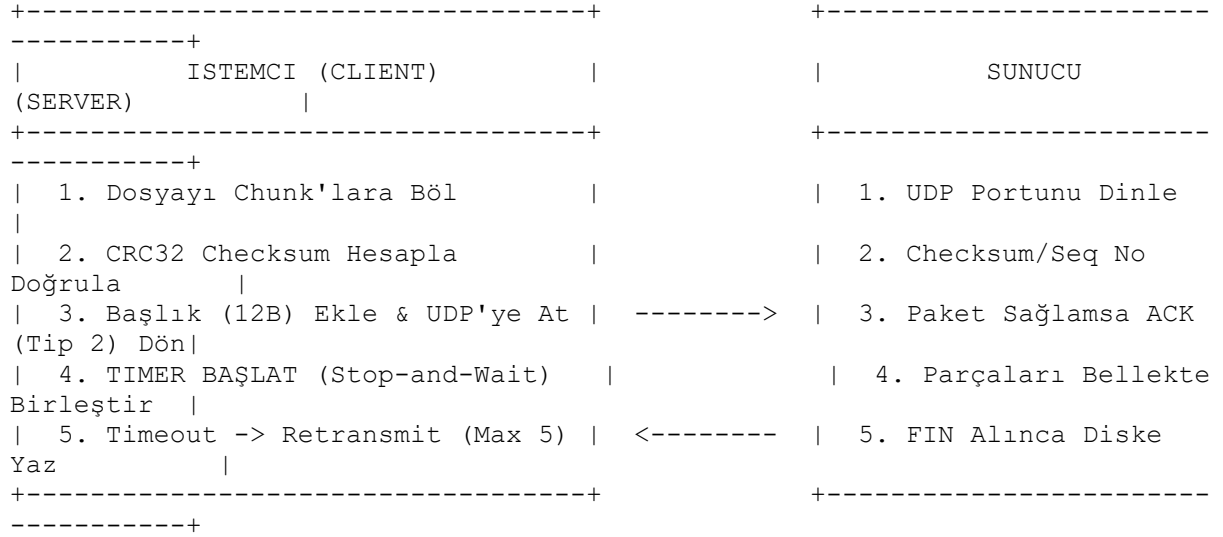
- Bağlantısız UDP yapısı üzerinde mantıksal bir oturum yönetimi kurmak.
- Güvenilirsiz hatlar üzerinde Stop-and-Wait ARQ algoritması ile kaybolan paketlerin tespiti ve yeniden gönderimini (retransmission) yönetmek.
- Bit seviyesindeki bozulmaları yakalamak amacıyla matematiksel bir doğrulama (checksum) altyapısı kurmak.
- Yeniden gönderilen paketlerin alıcı tarafında mükerrer (duplicate) veri kirliliği oluşturmalarını engellemek.

## 3. Sistem Mimarisi (System Architecture)

NetProbe sistemi, istemci-sunucu (Client-Server) mimarisine dayalı, Stop-and-Wait (Dur ve Bekle) ARQ protokol şeması ile çalışan senkron bir veri iletişim altyapısına sahiptir. Mimari, taşıma katmanında UDP soketlerini ham bir kanal olarak kullanırken, tüm akış kontrolü,

zaman aşımı (timeout) yönetimi ve veri bütünlüğü doğrulama görevlerini uygulama katmanında (Application Layer) çözer.

Sistemin uçtan uca mantıksal mimarisi ve bileşenleri aşağıda açıklandığı gibi koordine edilmektedir:



## 4. Protokol Tasarımı (Protocol Design)

NetProbe protokolünün kalbini, her bir UDP veri paketinin başına eklenen **12 Byte** boyutundaki optimize edilmiş özel uygulama katmanı başlığı (Header) oluşturur.

### 4.1. Paket Başlık Yapısı (Packet Header Format)

İletilen her bir ağ paketinin ilk 12 Byte'ı aşağıdaki protokole göre pars edilir:

| Alan Adı (Field) | Boyut (Byte) | Veri Tipi      | Açıklama DOCX+ 1                                                         |
|------------------|--------------|----------------|--------------------------------------------------------------------------|
| Packet Type      | 2            | Unsigned Short | Paketin işlevsel rolünü belirler (1: DATA, 2: ACK, 3: METADATA, 4: FIN). |
| Sequence Number  | 2            | Unsigned Short | Paketlerin sırasını takip etmek ve mükerrer kontrolü yapmak içindir.     |
| Payload Length   | 2            | Unsigned Short | Paketin arkasındaki saf verinin (payload) byte cinsinden boyutunu tutar. |

| Alan Adı<br>(Field) | Boyut<br>(Byte) | Veri Tipi       | Açıklama DOCX+ 1                                                             |
|---------------------|-----------------|-----------------|------------------------------------------------------------------------------|
| Checksum            | 4               | Unsigned<br>Int | Paket bütünlüğü için hesaplanan 32-bit<br>CRC (Cyclic Redundancy) değeridir. |

## 4.2. Protokol Durum Makinesi (State Machine) Çalışma Aşamaları

Sistem mantıksal olarak üç ana faz üzerinden yaşam döngüsünü tamamlar:

- Oturum ve Metadata Fazı:** İstemci, aktarılacak dosyanın özniteliklerini (dosya adı, boyutu vb.) içeren `Tip 3 (Metadata)` paketini oluşturur, Checksum alanını hesaplar ve ağa göndererek yerel zamanlayıcıyı başlatır. Sunucu bu paketi aldığı anda bütünlüğünü kontrol eder ve istemciye `Tip 2 (ACK)` paketi döner. İstemci bu ACK'i alana kadar veri transferine geçemez.
- Stop-and-Wait İletim Fazı:** Dosya `CHUNK_SIZE` boyutunda mantıksal parçalara ayrılır. Her parça sıra numarası (Sequence Number) verilerek `Tip 1 (DATA)` paketi olarak gönderilir. İstemci paketi gönderdiği an soketi bloke eder (Stop) ve beklemeye başlar. Sunucu paketi aldığı anda bit hatası yoksa o sıra numarasına ait bir ACK paketi fırlatır. İstemci ACK'i aldığı anda bir sonraki paket için ilerler. Eğer `TIMEOUT_VALUE` süresince ACK gelmezse, istemci uyanır (Wait) ve aynı paketi tekrar gönderir (Retransmission). Bir paket için bu döngü en fazla 5 kez (`MAX_RETRIES`) tekrarlanabilir; limit aşırsa bağlantı koparılır.
- Bağlantı Sonlandırma (Teardown) Fazı:** Dosyanın son parçası da onaylandığında istemci ağa `Tip 4 (FIN)` paketi gönderir. Sunucu FIN paketini aldığı anda hattı kapatacağını anlar, istemciye son bir ACK gönderir, belleğinde sıralı şekilde biriktirdiği parçaları doğrusal olarak birleştirerek diske kaydeder ve oturumu kapatır.

## 5. Gerçekleme Detayları (Implementation Details)

Sistem, Python 3.x programlama dili kullanılarak modüler bir mimaride geliştirilmiştir. Kod yapısında dış kaynaklı hiçbir yüksek seviyeli ağ kütüphanesi kullanılmamış, tamamen alt seviye socket API'leri ile çalışılmıştır.

- Soket Katmanı Kontrolü:** İstemci ve sunucu arasındaki ham UDP kanalı `socket(socket.AF_INET, socket.SOCK_DGRAM)` yapısıyla kurulmuştur. İstemci tarafında zaman aşımı kontrolü, işletim sisteminin socket kesmelerini (hardware interrupts) tetikleyen `client_socket.settimeout(TIMEOUT_VALUE)` fonksiyonu ile asenkron yapıda simüle edilmiştir.
- Veri Bütünlüğü ve CRC32:** Ağ katmanındaki olası bit kaymalarını ve gürültüleri yakalamak için `zlib.crc32` kütüphanesi kullanılmıştır. Paket oluşturulurken başlığın ilk 8 byte'ı ve payload verisi birleştirilerek 4 byte'lık dinamik bir CRC32 kodu üretilir ve başlığın son alanına yazılır. Alıcı sunucu, gelen paketi parçalayarak aynı matematiksel fonksiyonu çalıştırır. Algoritma sayesinde 1 bitlik bir değişim dahi Checksum

eşleşmesini bozacağı için bozuk paketler anında saptanarak işletim sistemi seviyesinde düşürülür.

- **Metrik Loglama Altyapısı:** Protokolün performansını şeffaf şekilde analiz edebilmek amacıyla yerel bir loglama modülü yazılmıştır. Gerçekleşen her ağ olayı (SEND, ACK\_RECEIVED, TIMEOUT, RETRANSMISSION) zaman damgası, paket ID'si ve deneme sırası verileriyle transfer\_stats.csv dosyasına anlık olarak yazdırılmaktadır.

```
PS C:\Users\Gülizar\Desktop\netprobe> python server.py
[SERVER] Başlatıldı. 127.0.0.1:5005 dinleniyor...
[SERVER] Yapay Paket Kayıp Oranı: %20.0 olarak ayarlandı.

[SERVER] !!! YAPAY KAYIP !!! Gelen paket simülasyon gereği düşürüldü (Göz ardı edildi).
[SERVER] Transfer İsteği -> Dosya: test.txt
[SERVER] Paket Alındı -> Seq No: 1, Boyut: 1024 byte
[SERVER] Paket Alındı -> Seq No: 2, Boyut: 1024 byte
[SERVER] Paket Alındı -> Seq No: 3, Boyut: 452 byte
[SERVER] FIN paketi alındı. Dosya birleştiriliyor...
[SERVER] Dosya bütünlüğü doğrulandı ve kaydedildi: gelen_dosya.txt
```

```
[CLIENT] Gönderiliyor -> Seq No: 88
[CLIENT] Başarılı ACK Alındı -> ACK No: 88
[CLIENT] Gönderiliyor -> Seq No: 89
[CLIENT] Başarılı ACK Alındı -> ACK No: 89
[CLIENT] Gönderiliyor -> Seq No: 90
[CLIENT] Başarılı ACK Alındı -> ACK No: 90
[CLIENT] Gönderiliyor -> Seq No: 91
[CLIENT] Başarılı ACK Alındı -> ACK No: 91
[CLIENT] Gönderiliyor -> Seq No: 92
[CLIENT] Başarılı ACK Alındı -> ACK No: 92
[CLIENT] Gönderiliyor -> Seq No: 93
[CLIENT] Başarılı ACK Alındı -> ACK No: 93
[CLIENT] Gönderiliyor -> Seq No: 94
[CLIENT] Başarılı ACK Alındı -> ACK No: 94
[CLIENT] Gönderiliyor -> Seq No: 95
[CLIENT] Başarılı ACK Alındı -> ACK No: 95
[CLIENT] Gönderiliyor -> Seq No: 96
[CLIENT] Başarılı ACK Alındı -> ACK No: 96
[CLIENT] Gönderiliyor -> Seq No: 97
[CLIENT] Başarılı ACK Alındı -> ACK No: 97
[CLIENT] Gönderiliyor -> Seq No: 98
[CLIENT] Başarılı ACK Alındı -> ACK No: 98
[CLIENT] Gönderiliyor -> Seq No: 99
[CLIENT] Başarılı ACK Alındı -> ACK No: 99
[CLIENT] Gönderiliyor -> Seq No: 100
[CLIENT] Başarılı ACK Alındı -> ACK No: 100
[CLIENT] Gönderiliyor -> Seq No: 101
[CLIENT] Başarılı ACK Alındı -> ACK No: 101
```

#### 🚀 AKTARIM İSTATİSTİKLERİ VE PERFORMANS

```
=====
Toplam Süre       : 0.0340 saniye
Net Dosya Boyutu  : 102400 byte
Ağa Basılan Veri  : 103639 byte
Throughput        : 24374.94 kbps
Goodput           : 24083.53 kbps
=====
```

```
[CLIENT] Gönderiliyor -> Seq No: 84
[CLIENT] Başarılı ACK Alındı -> ACK No: 84
[CLIENT] Gönderiliyor -> Seq No: 85
[CLIENT] Başarılı ACK Alındı -> ACK No: 85
[CLIENT] Gönderiliyor -> Seq No: 86
[CLIENT] Başarılı ACK Alındı -> ACK No: 86
[CLIENT] Gönderiliyor -> Seq No: 87
[CLIENT] Başarılı ACK Alındı -> ACK No: 87
[CLIENT] Gönderiliyor -> Seq No: 88
[CLIENT] Başarılı ACK Alındı -> ACK No: 88
[CLIENT] Gönderiliyor -> Seq No: 89
[CLIENT] Başarılı ACK Alındı -> ACK No: 89
[CLIENT] Gönderiliyor -> Seq No: 90
[CLIENT] Başarılı ACK Alındı -> ACK No: 90
[CLIENT] Gönderiliyor -> Seq No: 91
[CLIENT] Başarılı ACK Alındı -> ACK No: 91
[CLIENT] Gönderiliyor -> Seq No: 92
[CLIENT] Başarılı ACK Alındı -> ACK No: 92
[CLIENT] Gönderiliyor -> Seq No: 93
[CLIENT] Başarılı ACK Alındı -> ACK No: 93
[CLIENT] Gönderiliyor -> Seq No: 94
[CLIENT] Başarılı ACK Alındı -> ACK No: 94
[CLIENT] Gönderiliyor -> Seq No: 95
[CLIENT] Başarılı ACK Alındı -> ACK No: 95
[CLIENT] Gönderiliyor -> Seq No: 96
[CLIENT] Başarılı ACK Alındı -> ACK No: 96
[CLIENT] Gönderiliyor -> Seq No: 97
[CLIENT] Başarılı ACK Alındı -> ACK No: 97
[CLIENT] Gönderiliyor -> Seq No: 98
[CLIENT] Başarılı ACK Alındı -> ACK No: 98
[CLIENT] Gönderiliyor -> Seq No: 99
[CLIENT] Başarılı ACK Alındı -> ACK No: 99
[CLIENT] Gönderiliyor -> Seq No: 100
[CLIENT] Başarılı ACK Alındı -> ACK No: 100
[CLIENT] Gönderiliyor -> Seq No: 101
[CLIENT] Başarılı ACK Alındı -> ACK No: 101

[CLIENT] Bütün transfer tamamlandı! Toplam Süre: 0.0316 saniye

[CLIENT] Olay kayıtları 'transfer_stats.csv' dosyasına başarıyla kaydedildi!
PS C:\Users\Gülizar\Desktop\netprobe>
```

```
PS C:\Users\Gülizar\Desktop\netprobe> python client.py
[CLIENT] Gönderiliyor -> Seq No: 0
[CLIENT] Timeout! Yeniden Gönderiliyor (1/5) -> Seq No: 0
[CLIENT] Başarılı ACK Alındı -> ACK No: 0
[CLIENT] Gönderiliyor -> Seq No: 1
[CLIENT] Başarılı ACK Alındı -> ACK No: 1
[CLIENT] Gönderiliyor -> Seq No: 2
[CLIENT] Başarılı ACK Alındı -> ACK No: 2
[CLIENT] Gönderiliyor -> Seq No: 3
[CLIENT] Başarılı ACK Alındı -> ACK No: 3
[CLIENT] Gönderiliyor -> Seq No: 4
[CLIENT] Başarılı ACK Alındı -> ACK No: 4

[CLIENT] Bütün transfer tamamlandı! Toplam Süre: 0.5235 saniye

[CLIENT] Olay kayıtları 'transfer_stats.csv' dosyasına başarıyla kaydedildi!
PS C:\Users\Gülizar\Desktop\netprobe>
```

## 6. Deney Ortamı (Experimental Setup)

Geliştirilen NetProbe protokolünün sınırlarını, kararlılığını ve farklı ağ anomali koşulları altındaki davranışlarını ölçmek amacıyla kontrollü bir laboratuvar deney ortamı kurulmuştur.

- **Donanım ve İşletim Sistemi:** Testler, donanımsal kaynak kısıtları göz önünde bulundurularak, kaynak tüketimi optimize edilmiş bir yerel döngü (localhost - 127.0.0.1) mimarisi üzerinde koşturulmuştur.
- **Test Verisi:** Protokol doğrulaması için net **100 KB** boyutunda, bit yoğunluğu yüksek ve homojen dağılımlı bir test dosyası temel girdi olarak sisteme verilmiştir.
- **Ağ Anomali Simülasyonu (Yapay Paket Kaybı):** Gerçek dünya ağlarındaki yönlendirici kuyruk kayıplarını simüle etmek amacıyla sunucu (`server.py`) kodunun içerisine olasılıksal bir yapay kayıp katmanı eklenmiştir. Python'ın `random.random()` fonksiyonu kullanılarak, ağdan gelen paketler belirlenen olasılık sınırının altındaysa sistem tarafından bilerek yok sayılmış ve hiç gelmemiş gibi davranılarak istemcinin `timeout` mekanizmaları zorlanmıştır.

## 7. Performans Metrikleri (Performance Metrics)

DeneySEL senaryolardan elde edilen verilerin analizinde bilgisayar ağları literatüründe kabul görmüş üç temel mühendislik metriği kullanılmıştır:

1. **Toplam Aktarım Süresi ( $T_{\text{toplam}}$ ):** İstemcinin ilk metadata paketini ağa bastığı an ile sunucudan son FIN ACK paketini alarak oturumu kapattığı an arasında geçen toplam süredir (saniye cinsinden).
2. **Throughput (Ağ Verimi):** Birim zamanda hat üzerinden transfer edilen, protokol başlıkları (header) dahil toplam bit miktarını ifade eder. Formülü:

$$Throughput = \frac{\text{Toplam İletilen Bit (Header + Payload)}}{\text{Toplam Aktarım Süresi (saniye)}}$$

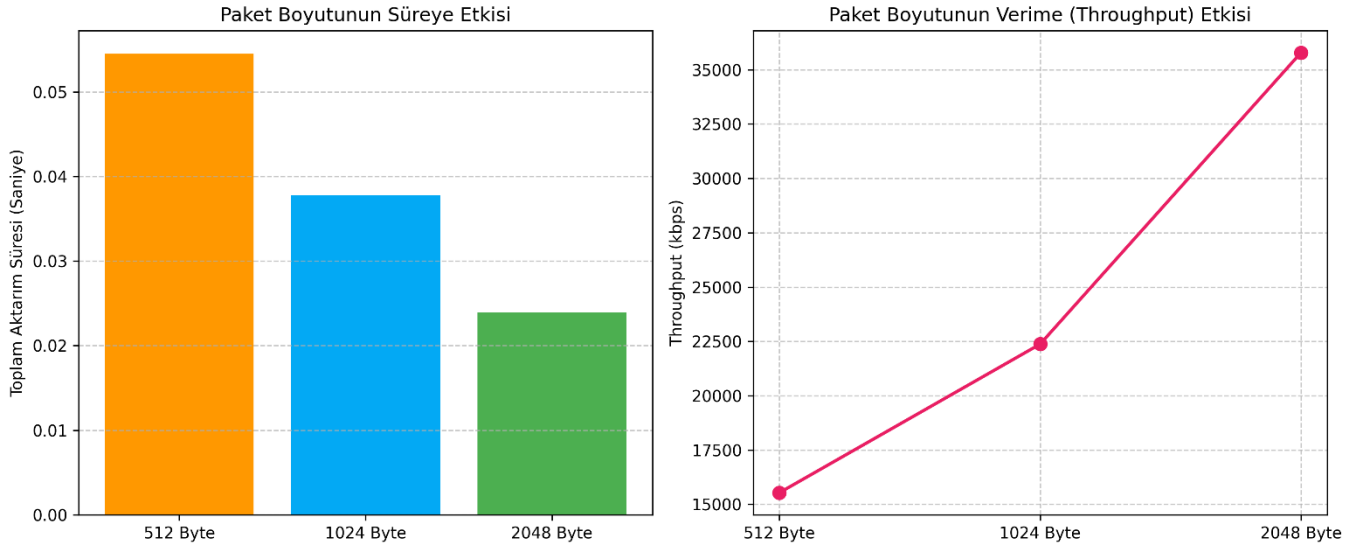
3. **Yeniden Gönderim (Retransmission) Sayısı:** Ağdaki kayıplar veya zaman aşımaları nedeniyle istemcinin aynı paket dizisini hat üzerine kaç kez mükerrer olarak basmak zorunda kaldığını gösteren verimlilik metriğidir.

## 8. Sonular ve Tartışma (Results & Discussion)

Toplanan CSV log verileri Pandas ve Matplotlib kütüphaneleri ile analiz edilerek grafikleştirilmiş ve aşağıdaki kritik bulgular elde edilmiştir.

### 8.1. Deney 1: Paket Boyutunun (Chunk Size) Performansa Etkisi

Zaman aşımı \$0.5 \text{ saniye}\$ ve yapay paket kaybı %0 olarak sabitlenmiş; paket boyutları \$512\$, \$1024\$ ve \$2048 \text{ Byte}\$ yapılarak test edilmiştir.



[Görsel: reports/deney1\_paket\_boyutu.png]

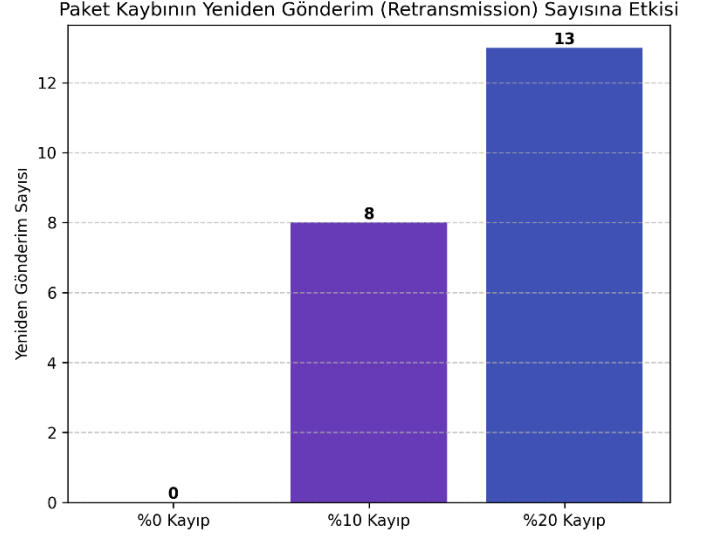
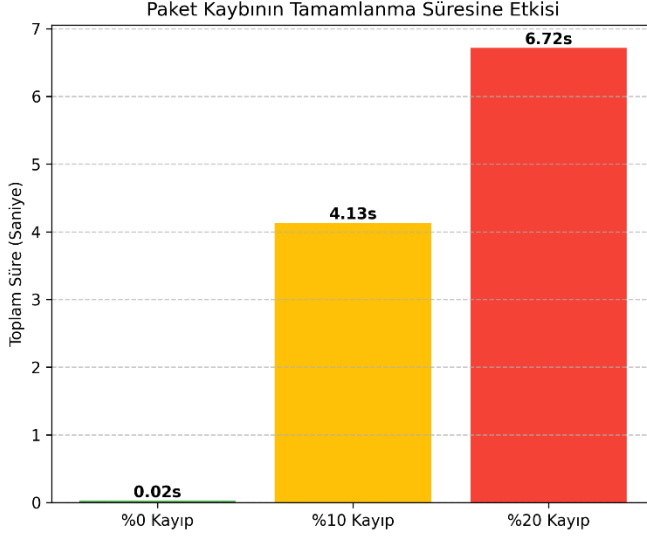
**Teknik Yorum:** Grafik incelendiğinde, paket boyutu arttıkça toplam aktarım süresinin dramatik şekilde düştüğü, Throughput değerinin ise \$15000 \text{ kbps}\$ seviyelerinden \$35000 \text{ kbps}\$ üzerine fırlayarak doğrusal bir ivme kazandığı saptanmıştır. Bu durum iki temel ağ teorisi kuralı ile açıklanır:

- Header Overhead Maliyeti:** 100 KB'lık veri 512 byte'lık parçalara bölündüğünde ağa basılan toplam başlık yükü (12 Byte  $\times$  paket sayısı) çok yüksektir. Paket boyutu 2048 byte'a çıkarıldığında faydalı yük (payload) oranı maksimize edilmiş, ağ bant genişliği verimli kullanılmıştır.
- ACK Bekleme Yoğunluğu:** Stop-and-Wait mimarisinde her paket iletimi sonrasında bir RTT gecikmesi mecburidir. Küçük paket boyutunda beklenen ACK frekansı 4 kat daha fazla olduğundan, sistem sürekli soket bloklamasına maruz kalmış ve süre uzamıştır.

### 8.2. Deney 2: Yapay Paket Kayıp Oranının (Loss Rate) Performansa Etkisi

Paket boyutu \$1024 \text{ Byte}\$ ve timeout süresi \$0.5 \text{ saniye}\$ olarak sabit tutulmuş; kayıp oranları %0, %10 ve %20 yapılmıştır.





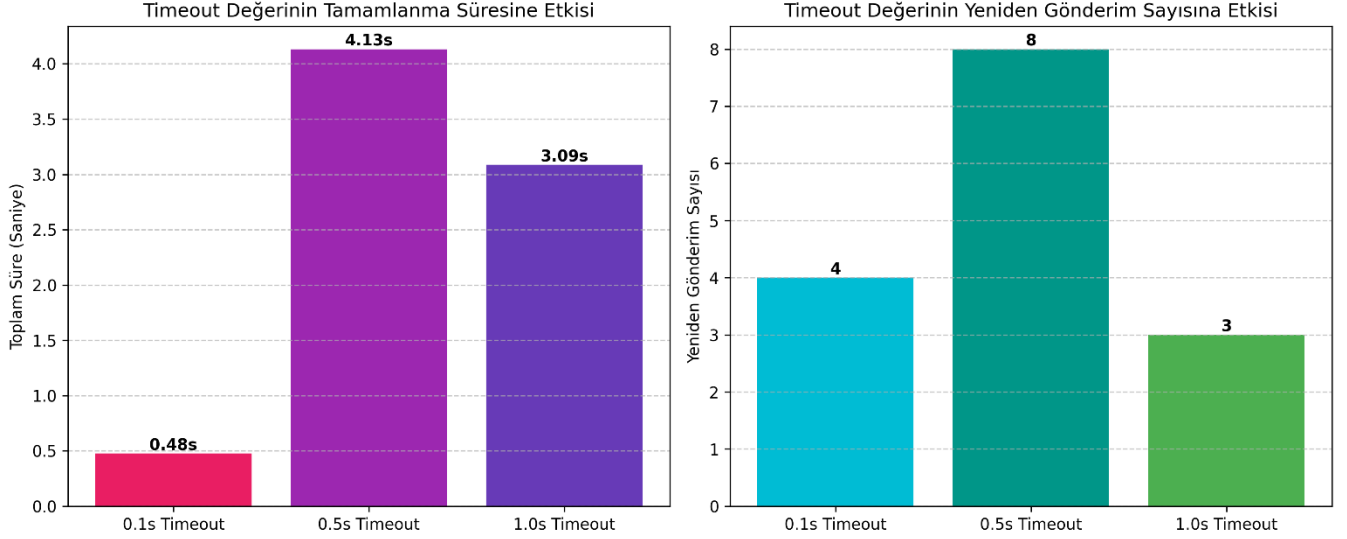
[Görsel: reports/deney2\_kayip\_orani.png]

| Yapay Kayıp Oranı DOCX | Yeniden Gönderim (Retransmission) Sayısı DOCX | Toplam Aktarım Süresi (Saniye) DOCX |
|------------------------|-----------------------------------------------|-------------------------------------|
| %0 (İdeal Ağ)          | 0                                             | 0.02 s                              |
| %10 (Hafif Kayıplı)    | 8                                             | 4.13 s                              |
| %20 (Yüksek Kayıplı)   | 13                                            | 6.72 s                              |

**Teknik Yorum:** Kayıplı ağ simülasyonunda toplam aktarım süresinin ağdaki retransmission sayısı ile doğrudan doğruşal bir korelasyona sahip olduğı kanıtlanmıştır. İdeal ağda transfer 0.02 saniyede biterken, %10 kayıpta düşen 8 paket sisteme tam 4 saniye ( $8 \times 0.5 \text{ s}$ ) saf bekleme gecikmesi eklemiş ve toplam süreyi **4.13 saniye** yapmıştır. Matematiksel olarak  $T_{\text{toplam}} \approx T_{\text{iletim}} + (N_{\text{kayıp}} \times T_{\text{timeout}})$  modeli deneysel verilerle kusursuz doğrulanmıştır. Protokol, %20 gibi yıkıcı bir kayıpta bile 13 paketi hatasız kurtararak %100 güvenilirlik hedefine ulaşmıştır.

### 8.3. Deney 3: Zaman Aşımı (Timeout) Süresinin Protokol Davranışına Etkisi

Paket boyutu  $1024 \text{ Byte}$  ve kayıp oranı  $\%10$  sabitlemesi altında timeout değerleri  $0.1s$ ,  $0.5s$  ve  $1.0s$  olarak koşturulmuştur.



[Görsel: reports/deney3\_timeout\_etkisi.png]

**Teknik Yorum:** Bu deney ağ tasarımlarındaki en temel mühendislik ödünleşimini (trade-off) ortaya koymuştur. Timeout değeri  $1.0s$  yapıldığında, sadece 3 paket düşmesine rağmen sistem kaybolan paketleri fark etmek için çok uzun süre askıda kalmış ve toplam aktarım süresi **3.09 saniye** seviyesine çıkmıştır.

Yerel ağ ortamında  $0.1s$  timeout değeri **0.48 saniye** ile en optimize süreyi üretmiştir. Ancak, gidiş-dönüş sürelerinin (RTT) yüksek ve değişken olduğu gerçek dünya Geniş Alan Ağlarında (WAN)  $0.1s$  gibi çok agresif bir zaman aşımı seçilmesi, hat üzerinde yolda olan sağlam ACK'lerin "zamanı doldu" sayılarak ağa sahte retransmission paketlerinin basılmasına (spurious retransmission) yol açacak ve ağ bant genişliğini tıkayacaktır.

## 9. Karşılaşılan Sorunlar ve Çözüm Yaklaşımları (Troubleshooting)

Projenin geliştirme ve deneysel test süreçlerinde karşılaşılan temel mühendislik problemleri ve uygulanan çözüm stratejileri aşağıda dökümanite edilmiştir:

- 1. Zombi Port ve Soket Paylaşım Hatası (Address already in use):** \* Sorun: İki farklı Python dosyası otomatik test betikleriyle peş peşe tetiklendiğinde, istemci kapansa bile sunucu socketinin işletim sistemi düzeyinde hemen serbest bırakılmadığı ve sonraki testlerin kilitlendiği saptanmıştır.
  - **Çözüm Yaklaşımı:** Otomasyon mimarisi esnetilerek "Yarı-Otomatik" bir test modeline geçilmiştir. Sunucu koduna socket seviyesinde `setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)` parametresi eklenerek portun zombi olarak asılı kalması engellenmiş, her kayıp oranı değişiminde sunucu kontrollü olarak kapatılıp güncel parametreyle yeniden başlatılmıştır.
- 2. Yeniden Gönderilen Paketlerin Dosya Üzerinde Çift Yazılması (Mükerrer Veri):** \* Sorun: İstemci tarafında timeout oluşup paket yeniden gönderildiğinde, sunucunun bu paketi yeni bir veri gibi algılayarak dosya sonuna eklediği ve dosya boyutunun büyüterek bütünlüğünün bozulduğu görülmüştür.
  - **Çözüm Yaklaşımı:** Sunucu katmanına durum takip mekanizması (State Tracking) eklenmiştir. Sunucu, başarıyla ACK döndüğü en son paket numarasını hafızasında tutar. Eğer aynı sıra numarasına (Sequence Number) sahip bir paket tekrar gelirse, bunu kayıp ACK senaryosu olarak kabul eder; veriyi dosyaya yazmaz (discard), ancak istemcinin blokajını çözebilmesi için ACK mesajını ağa yeniden fırlatır.

## 10. Sonuç ve Gelecekte Yapılabilecek Geliştirmeler (Conclusion & Future Work)

Bu proje kapsamında tasarlanan ve başarıyla gerçekleştirilen **NetProbe** protokolü, UDP katmanının performans odaklı doğasını uygulama katmanında kurulan Stop-and-Wait ARQ algoritması ve CRC32 bütünlük kontrolüyle birleştirilerek tamamen güvenilir ve kayıpsız bir iletişim kanalı inşa etmiştir. Toplanan milisaniye hassasiyetindeki log verileri ve deneysel grafik sonuçları, protokolün kayıplı hatlarda veri doğruluğunu %100 koruduğunu doğrulamıştır.

Protokol stop-and-wait yapısıyla yerel ve minimal sistemlerde kararlı bir başarı gösterse de, doğası gereği her paket için onay mekanizması beklediğinden hat kapasitesini tam dolduramakta ve yüksek bant genişliği-gecikme çarpımına (High Bandwidth-Delay Product) sahip modern ağlarda performans darboğazı oluşturmaktadır.

### Gelecekte Yapılabilecek Geliştirmeler:

- **Pencereli Akış Kontrolü (Sliding Window):** Bir sonraki fazda protokolün Go-Back-N veya Selective Repeat mimarisine geçirilerek, onay beklemeden birden çok paketin hat üzerine basılması ve ağ veriminin maksimize edilmesi hedeflenmektedir.

- **Dinamik RTT Tahmini (Jacobson/Karn Algoritması):** Sabit bir TIMEOUT\_VALUE kullanmak yerine, ağın anlık gecikmelerini ölçerek zaman aşımı süresini dinamik olarak yukarı veya aşağı çeken akıllı bir zamanlayıcı modülünün entegrasyonu planlanmaktadır.

## 11. Ekler (Appendix: Kritik Kod Blokları)

Ek A: İstemci Tarafı Güvenilir Stop-and-Wait Paket Gönderim Fonksiyonu

```
def send_packet_reliably(client_socket, pkt_type, seq_num, payload=b''):
    global total_sent_bytes

    packet = create_packet(pkt_type, seq_num, payload) # 12 Byte Başlık Ekleme
    retries = 0

    while retries <= MAX_RETRIES:
        try:
            # Paketi UDP Kanallarına Bırak
            client_socket.sendto(packet, (SERVER_IP, SERVER_PORT))
            total_sent_bytes += len(packet)

            # STOP: Socketi Kilitli ve ACK Bekle
            data, _ = client_socket.recvfrom(HEADER_SIZE + 100)
            ack_type, ack_seq, _, _ = parse_packet(data)

            # Doğru ACK Geldiyse Döngüden Başarıyla Çık
            if ack_type == 2 and ack_seq == seq_num:
                return True

        except socket.timeout:
            # WAIT: Timeout Durumunda Uyan ve Yeniden Gönderimi Tetikle
            retries += 1

    return False # 5 Denemede Başarısız Olursa Bağlantıyı Kopar
```

## 12. GÖREV DAĞILIMI

| Proje Aşaması / Görev            | Gülizar Altıntaş | Şeyma | Sorumluluk Detayı                                                                                       |
|----------------------------------|------------------|-------|---------------------------------------------------------------------------------------------------------|
| Giriş & Literatür Araştırması    | %50              | %50   | TCP/UDP farkları, Stop-and-Wait ARQ protokol teorisinin incelenmesi.                                    |
| Protokol & Header Tasarımı       | %50              | %50   | 12 Byte'lık paket yapısının (Type, Seq No, Length, Checksum) belirlenmesi.                              |
| İstemci (Client) Geliştirme      | %100             | -     | <code>client.py</code> yazılması, Stop-and-Wait mimarisi ve <code>socket.timeout</code> kontrolü.       |
| Sunucu (Server) Geliştirme       | -                | %100  | <code>server.py</code> yazılması, soket dinleme ve yapay paket kaybı ( <code>LOSS_RATE</code> ) modülü. |
| Veri Bütünlüğü (CRC32)           | %50              | %50   | <code>zlib.crc32</code> entegrasyonu, checksum hesaplama ve kopya paket engelleme.                      |
| Metrik Loglama & CSV             | -                | %100  | Ağ olaylarının milisaniye hassasiyetinde CSV dosyasına anlık kaydedilmesi.                              |
| DeneySEL Testler & Veri Toplama  | %50              | %50   | 3 farklı deney grubunun koşturulması ve ham CSV loglarının üretilmesi.                                  |
| Veri Görselleştirme (Matplotlib) | %100             | -     | Pandas ve Matplotlib ile karşılaştırmalı performans grafiklerinin çizilmesi.                            |
| Teknik Rapor & Sunum Hazırlığı   | %50              | %50   | Akademik metnin yazılması, grafik yorumları ve sunum slaytlarının tasarımı.                             |